Inatel

Laboratório de Microcontroladores

Semana 02 - Revisão de Algoritmos

Monitor: Gustavo Pivoto Ambrósio

Linguagem C

- A linguagem C foi criada por Dennis Ritchie em 1972 no Bell Labs.
- É uma linguagem de programação compilada, estruturada e de propósito geral.
- Amplamente utilizada no desenvolvimento de sistemas operacionais, sistemas embarcados e aplicações que exigem alta performance, pois ela permite manipulação de baixo nível e controle direto da memória, sendo uma linguagem fundamental na computação.

Variáveis e Tipos de Dados

- Uma variável é um local reservado na memória para armazenar um tipo de dado.

- Toda variável possui um identificador (nome) e um tipo de dado. O tipo determina quais valores a variável pode armazenar e quais operações podem ser realizadas com ela.

- São tipos de dados comuns em linguagem C:
  - int: armazena valores numéricos inteiros.
  - char: armazena caracteres.
  - float: armazena números com ponto flutuante (reais) com precisão simples.
  - double: armazena números com ponto flutuante, com precisão dupla.

Variáveis e Tipos de Dados

Exemplo:

```c
#include <stdio.h>

int main()
{
    int idade = 21;         // Declara uma variável inteira e atribui o valor 21
    float altura = 1.75;    // Declara uma variável de ponto flutuante e atribui o valor 1.75

    printf("Olá! Tenho %d anos e %.2f metros de altura.\r\n", idade, altura);

    return 0;
}
```

Operadores

- Um operador é um símbolo que representa uma operação a ser realizada sobre uma ou mais variáveis ou valores.
- A linguagem C possui diversos operadores, sendo os principais apresentados a seguir.

Operadores Aritméticos

- Os operadores aritméticos são utilizados para realizar operações matemáticas sobre valores numéricos.

Operador	Operação
+	Soma
-	Subtração
*	Multiplicação
/	Divisão
%	Resto da divisão
++	Incremento
--	Decremento

Operadores Aritméticos

Exemplo:

```c
#include <stdio.h>

int main(void) {

    int a = 20, b = 5;

    int soma = a + b;
    printf("Soma: %d\n", soma); // 25

    int subtracao = a - b;
    printf("Subtração: %d\n", subtracao); // 15

    int mult = a * b;
    printf("Multiplicação: %d\n", mult); // 100

    int divisao = a / b;
    printf("Divisão: %d\n", divisao); // 4

    int mod = a % b;
    printf("Resto: %d\n", mod); // 0

    a++;
    printf("Incremento: %d\n", a); // 21

    b--;
    printf("Decremento: %d\n", b); // 4

    return 0;
}
```

Operadores Relacionais

- Os operadores relacionais são usados para comparar dois valores e produzir um valor lógico (verdadeiro ou falso) como resultado.

Operador	Operação
==	Igual
!=	Diferente
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

Operadores Relacionais

Exemplo:

```c
#include <stdio.h>

int main(void) {

    int a = 20, b = 5;

    int resultado = (a == b);
    printf("%d\n", resultado); // 0

    resultado = (a != b);
    printf("%d\n", resultado); // 1

    resultado = (a > b);
    printf("%d\n", resultado); // 1

    resultado = (a < b);
    printf("%d\n", resultado); // 0

    resultado = (a >= b);
    printf("%d\n", resultado); // 1

    resultado = (a <= b);
    printf("%d\n", resultado); // 0

    return 0;
}
```

Operadores Lógicos

- Os operadores lógicos são utilizados para combinar expressões relacionais, produzindo um resultado lógico (verdadeiro ou falso).

Operador	Operação
&&	AND
||	OR
!	NOT

Operadores Lógicos

Exemplo:

```c
#include <stdio.h>

int main(void) {

    int a = 1, b = 0;

    int resultado = (a && b);
    printf("%d\n", resultado); // 0

    resultado = (a || b);
    printf("%d\n", resultado); // 1

    resultado = !a;
    printf("%d\n", resultado); // 0

    return 0;
}
```

Operadores Bit a Bit

- Os operadores bit a bit são utilizados para manipular individualmente os bits de um valor numérico, sendo especialmente úteis quando se trabalha diretamente com representações binárias.

Operador	Operação
&	AND
|	OR
~	NOT
^	XOR
<<	Deslocamento de bits à esquerda
>>	Deslocamento de bits à direita

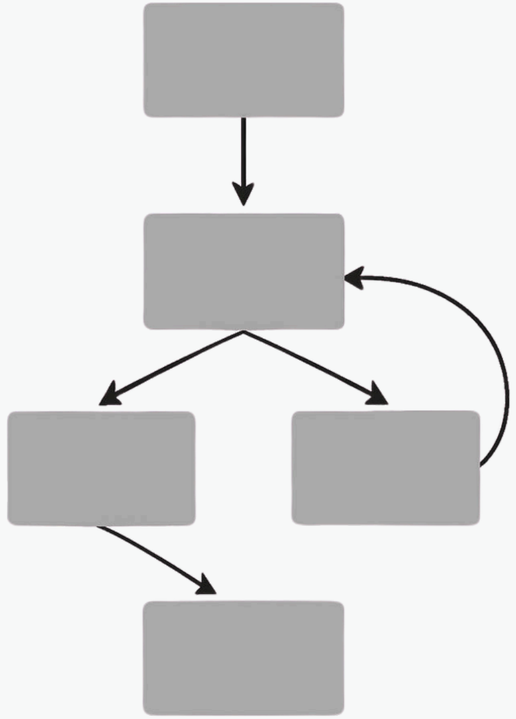
Operadores Bit a Bit

Exemplo:

```c
#include <stdio.h>

int main(void) {

    unsigned char a = 6;  // 00000110
    unsigned char b = 2;  // 00000010
    unsigned char resultado;

    // AND bit a bit
    resultado = a & b;
    printf("a & b = %d\n", resultado);  // 2

    // OR bit a bit
    resultado = a | b;
    printf("a | b = %d\n", resultado);  // 6

    // XOR (OU exclusivo)
    resultado = a ^ b;
    printf("a ^ b = %d\n", resultado);  // 4

    // Negação bit a bit
    resultado = ~a;
    printf("~a = %d\n", resultado);  // 249 (em 8 bits)

    // Deslocamento à esquerda
    resultado = a << 1;
    printf("a << 1 = %d\n", resultado);  // 12

    // Deslocamento à direita
    resultado = a >> 1;
    printf("a >> 1 = %d\n", resultado);  // 3

    return 0;
}
```

Estruturas de Controle de Fluxo

- Estruturas de controle de fluxo determinam a ordem em que as instruções de um programa são executadas, possibilitando decisões e repetições com base em condições.



Estruturas de Controle de Fluxo

Exemplo:

```c
#include <stdio.h>

int main(void) {
    int idade;

    scanf("%d", &idade); // 15

    if(idade < 18) {
        printf("Você é menor de idade.\n"); // <-
    } else {
        printf("Você é maior de idade.\n");
    }

    return 0;
}
```

if - else

É usada para verificar condições e executar blocos de código diferentes dependendo do resultado.

Estruturas de Controle de Fluxo

switch - case

É usada para avaliar uma variável ou expressão e executar diferentes blocos de código com base no valor correspondente entre múltiplas opções (cases).

Exemplo:

```c
#include <stdio.h>

int main(void) {
    char opcao;

    scanf("%c", &opcao); // D

    switch (opcao) {
        case 'A':
            printf("Opção A escolhida.\n");
            break;
        case 'B':
            printf("Opção B escolhida.\n");
            break;
        case 'C':
            printf("Opção C escolhida.\n");
            break;
        default:
            printf("Opção inválida.\n"); // <-
    }

    return 0;
}
```

Estruturas de Controle de Fluxo

Exemplo:

while

É utilizada para repetir um bloco de código enquanto uma condição for verdadeira.

```c
#include <stdio.h>

int main(void) {
    int contador = 1;

    while (contador <= 10) {
        printf("%d ", contador);

        contador++;
    }

    return 0;
}
```

Estruturas de Controle de Fluxo

Exemplo:

for

É usada para repetir um bloco de código por um número determinado de vezes.

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i < 11; i++) {
        printf("%d ", i);
    }

    return 0;
}
```

Funções

- Funções são usadas para modularizar o código, tornando-o mais organizado, legível e reutilizável.
- São blocos de código que executam uma tarefa específica.
- Funções podem receber parâmetros, que são valores passados para a função no momento da chamada, permitindo que ela opere sobre dados externos.
- Toda função em C possui um tipo de retorno, que define o tipo de dado que ela devolverá ao final da execução. Caso não retorne nenhum valor, utiliza-se o tipo void.

Funções

Exemplos:

```c
#include <stdio.h>

int soma(int x, int y) {
    return x + y;
}

int main(void) {
    int x = 10;
    int y = 5;

    int resultado = soma(x, y);

    printf("%d + %d = %d\n", x, y, resultado); // 15

    return 0;
}
```

```c
#include <stdio.h>

void soma(int x, int y) {
    int resultado = x + y;
    printf("%d + %d = %d\n", x, y, resultado);
}

int main(void) {
    int x = 5;
    int y = 2;

    soma(x, y); // 7

    return 0;
}
```

Inatel

Dúvidas?